SATS User Guide

Version: SATS 1.0.9

SATS (Signature Analyzer for Targeted Sequencing) is an R package for mutational signature analysis in targeted sequencing data. The method models panel-specific mutation opportunity, so it can be used for de novo signature detection, mapping of de novo profiles to tumor mutational burden (TMB)-normalized reference signatures, signature refitting in individual tumors, and calculation of signature-attributed mutation burdens.

The accompanying manuscript applies SATS to 111,711 tumors from AACR Project GENIE to construct a real-world, panel-calibrated pan-cancer catalogue of targeted sequencing-derived mutational signatures. The package is intended for targeted-panel cohorts where whole-exome or whole-genome sequencing data are unavailable or not routinely generated.

Installation

The current source version in this repository is SATS v1.0.9. The recommended installation route is from the GitHub source tree:

```
if (!requireNamespace("devtools", quietly = TRUE))
    install.packages("devtools")
devtools::install_github("binzhulab/SATS", subdir = "source", upgrade = "never")
```

Alternatively, download `SATS_1.0.9.tar.gz` from the repository and install the source archive:

```
R CMD INSTALL SATS_1.0.9.tar.gz
```

The source archive can also be installed from within R:

```
install.packages("./SATS_1.0.9.tar.gz", repos = NULL, type = "source")
```

SATS was formerly available from CRAN. CRAN currently lists the package as archived, so the GitHub source installation above is the recommended installation route for the current version. After installation, load the package with:

```
library(SATS)
```

The de novo signature discovery examples use `signeR`, which should be installed separately if that step is run locally. SATS can still be used for preprocessing, signature mapping, refitting and burden calculation without running `signeR`.

Required Inputs

SATS uses three main input matrices. The mutation catalogue matrix `V` has dimension `P x N`, where rows are mutation contexts and columns are samples. For SBS analysis, `P = 96`. The panel-context matrix `L` has the same dimension as `V` and gives the number of mutation opportunities per million base pairs for each mutation context and sample. The reference signature matrix `W` has dimension `P x K`, where columns are TMB-normalized reference signatures.

The row order of `V`, `L` and `W` must match. For SBS analyses, SATS supports the COSMIC-style 96-channel order and the `signeR` order. The `SBS_order` argument in `GeneratePanelSize()` controls mutation-type ordering only; it does not select the COSMIC reference-signature version. The reference-signature version used by `MappingSignature()` is controlled separately by `COSMICv`, with `"v3.4"` as the current default.

SATS accepts either summarized mutation-count and panel-context matrices or MAF-like mutation-record tables with panel-coordinate data frames. It does not directly ingest raw VCF or BED files. VCF/BED-derived data should first be converted into MAF-like mutation records and panel-coordinate tables before running SATS.

Example Data

The package includes simulated data in the `SimData` object:

```
data(SimData, package = "SATS")
names(SimData)
```

`SimData$V` is a simulated 96 x 10027 SBS mutation catalogue matrix and `SimData$L` is the corresponding panel-context matrix. `SimData$TrueW_TMB` and `SimData$TrueH` are the simulated TMB-normalized signature profiles and activity matrix used to generate `SimData$V`. `SimData$PanelEx` and `SimData$PatientInfo` provide example panel and sample annotation data for constructing an `L` matrix.

Generating V and L from MAF-like Mutation Records

`GenerateVMatrix()` generates an SBS96 or DBS78 mutation-count matrix from a MAF-like mutation record table. The mutation record must contain `Chromosome`, `Start_Position`, `End_Position`, `Variant_Type`, `Reference_Allele`, `Tumor_Seq_Allele2` and `Tumor_Sample_Barcode`. SBS analyses use records with `Variant_Type == "SNP"` and DBS analyses use records with `Variant_Type == "DNP"`.

```
dir <- system.file("extdata", "refitting_examples", package = "SATS")
sbs_file <- file.path(dir, "SBS_MAF_two_samples.txt")
sbs_mut <- read.table(sbs_file, header = TRUE, sep = "\t", quote = "",
                      stringsAsFactors = FALSE)

V_sbs <- GenerateVMatrix(sbs_mut, Class = "SBS", ref.genome = "hg19")
dim(V_sbs)
```

`GenerateLMatrix()` prepares the matched panel-context matrix from a panel-coordinate table and a clinical sample table linking samples to sequencing assays. The clinical sample table must contain `SEQ_ASSAY_ID` and either `SAMPLE_ID` or `PATIENT_ID`. This two-function preprocessing workflow prepares input matrices for downstream SATS analysis but does not select cancer-type-specific signatures or run refitting automatically.

```
data(SimData, package = "SATS")

clinical_sample <- data.frame(
    SAMPLE_ID = unique(sbs_mut$Tumor_Sample_Barcode),
    SEQ_ASSAY_ID = SimData$PatientInfo$SEQ_ASSAY_ID[1],
    stringsAsFactors = FALSE
)

L_sbs <- GenerateLMatrix(
    Panel_context = SimData$PanelEx,
    Patient_Info = clinical_sample,
    Class = "SBS",
    SBS_order = "COSMIC",
    ref.genome = "hg19"
)

if (!setequal(colnames(V_sbs), colnames(L_sbs)))
    stop("V and L contain different sample IDs")
if (!setequal(rownames(V_sbs), rownames(L_sbs)))
    stop("V and L contain different mutation-context rows")

L_sbs <- L_sbs[rownames(V_sbs), colnames(V_sbs), drop = FALSE]
identical(rownames(V_sbs), rownames(L_sbs))
identical(colnames(V_sbs), colnames(L_sbs))
```

Generating the Panel-Context Matrix

`GeneratePanelSize()` calculates panel-level mutation-context opportunity counts from panel-coordinate information. The input data frame must contain the columns `Chromosome`, `Start_Position`, `End_Position` and `SEQ_ASSAY_ID`. `Chromosome`, `Start_Position` and `End_Position` define the genomic interval, and `SEQ_ASSAY_ID` identifies the sequencing panel.

```
data(SimData, package = "SATS")

Panel_context <- GeneratePanelSize(
    genomic_information = SimData$PanelEx,
    Class = "SBS",
    SBS_order = "COSMIC",
    ref.genome = "hg19"
)
```

`Class` can be `"SBS"` or `"DBS"`. For SBS analysis, `SBS_order` can be `"COSMIC"` or `"signeR"`. `ref.genome` can be `"hg19"` or `"hg38"` and requires the corresponding Bioconductor reference-genome package.

`GenerateLMatrix()` converts panel-coordinate information or panel-level context counts into a sample-level `L` matrix by matching patient identifiers to sequencing panels. The `Patient_Info` data frame must contain `SEQ_ASSAY_ID` and either `PATIENT_ID` or `SAMPLE_ID`.

```
PatientInfo <- SimData$PatientInfo[
    SimData$PatientInfo$SEQ_ASSAY_ID %in% unique(SimData$PanelEx$SEQ_ASSAY_ID),
]

L_mat <- GenerateLMatrix(Panel_context, PatientInfo)
dim(L_mat)
```

The resulting `L` matrix is used as the opportunity matrix for `signeR()` and as the panel-context matrix for `EstimateSigActivity()` and `CalculateSignatureBurdens()`.

De Novo Signature Detection and Mapping

For cohort-level signature detection, SATS can be used with de novo profiles estimated by `signeR` or another compatible signature extraction method. After `V_mat` and `L_mat` have been generated and aligned, choose the initial discovery strategy according to cohort size. If the sample size is small, for example fewer than 100 samples, the individual matched samples can be used directly. If the cohort is very large, such as a real-world cohort with about 10,000 tumors, every 100 matched samples can be pooled into one profile before de novo discovery. SATS stores mutation contexts in rows and samples in columns, whereas `signeR()` expects samples in rows and mutation contexts in columns; therefore, the matched SATS matrices are transposed when passed to `signeR()`. In the pooled workflow, `V_sum` and `L_sum` are derived by summing the same sample columns of `V_mat` and `L_mat`; they are not separate input files. In a full analysis, `W_hat` is the de novo TMB-normalized signature profile matrix returned by the extraction step. The examples below use simulated package matrices so that the code can be run end to end.

```
data(SimData, package = "SATS")

V_mat <- SimData$V
L_mat <- SimData$L
stopifnot(identical(rownames(V_mat), rownames(L_mat)))
stopifnot(identical(colnames(V_mat), colnames(L_mat)))

library(signeR)

# Example 1: use individual matched samples directly for a small cohort.
# If more than 100 samples are available, this example uses the first 100 only
# to keep the demonstration compact.
n_example <- min(100L, ncol(V_mat))
sample_idx <- seq_len(n_example)
V_fit <- V_mat[, sample_idx, drop = FALSE]
L_fit <- L_mat[, sample_idx, drop = FALSE]
stopifnot(identical(rownames(V_fit), rownames(L_fit)))
stopifnot(identical(colnames(V_fit), colnames(L_fit)))

set.seed(1)
signeR_re_100 <- signeR(M = t(V_fit), Opport = t(L_fit), nlim = c(1, 5))
W_hat_100 <- signeR_re_100$Phat
stopifnot(identical(rownames(W_hat_100), rownames(V_fit)))

# Example 2: pool every 100 matched samples for a very large cohort.
# A cohort with about 10,000 tumors would yield about 100 pooled profiles.
pool_id <- ceiling(seq_len(ncol(V_mat)) / 100)
V_sum <- t(rowsum(t(V_mat), group = pool_id, reorder = FALSE))
L_sum <- t(rowsum(t(L_mat), group = pool_id, reorder = FALSE))
stopifnot(identical(rownames(V_sum), rownames(L_sum)))
stopifnot(identical(colnames(V_sum), colnames(L_sum)))

set.seed(1)
signeR_re_pool <- signeR(M = t(V_sum), Opport = t(L_sum), nlim = c(1, 5))
W_hat <- signeR_re_pool$Phat
stopifnot(identical(rownames(W_hat), rownames(V_sum)))
```

The de novo TMB-based profiles are then mapped to TMB-normalized reference signatures using `MappingSignature()`:

```
data(RefTMB, package = "SATS")

MappedSig <- MappingSignature(
    W_hat = W_hat,
    W_ref = RefTMB$TMB_SBS_v3.4
)
MappedSig
```

If `W_ref` is not supplied, `MappingSignature()` defaults to `COSMICv = "v3.4"` and uses `RefTMB$TMB_SBS_v3.4`. The returned data frame reports the selected reference signatures and the frequency with which each signature is selected across repeated penalized non-negative least-squares fits.

Estimating Signature Activities

After defining the reference signatures to use for refitting, estimate signature activities with `EstimateSigActivity()`.

```
data(SimData, package = "SATS")
data(RefTMB, package = "SATS")

SBS.list <- c("SBS1", "SBS2_13", "SBS4", "SBS5", "SBS6", "SBS89")
W_star <- as.matrix(RefTMB$TMB_SBS_v3.4[, SBS.list])

H_hat <- EstimateSigActivity(
    V = SimData$V,
    L = SimData$L,
    W = W_star
)
H_hat$H
```

`EstimateSigActivity()` uses an expectation-maximization algorithm and returns a list containing the estimated activity matrix `H`, the log-likelihood and a convergence flag. The estimated activity matrix has dimension `K x N`.

Calculating Signature Burdens

Signature burdens are the expected numbers of mutations attributed to each selected signature in each tumor. They are calculated with `CalculateSignatureBurdens()`:

```
SigBdn <- CalculateSignatureBurdens(
    L = SimData$L,
    W = W_star,
    H = H_hat$H
)

round(SigBdn[, 1:5], 2)
```

The returned matrix has dimension `K x N`, with signatures in rows and samples in columns.

Single-Tumor or Small-Cohort Refitting

SATS can also refit a prespecified signature set in a single tumor or a small cohort. In this setting, the reference set should be constrained to signatures that are relevant to the tumor type and detectable in targeted sequencing data. `RefTMB$SBS_refSigs` and `RefTMB$DBS_refSigs` provide cancer-specific SBS and DBS reference-signature lists.

```
data(SimData, package = "SATS")
data(RefTMB, package = "SATS")

SBS.list <- RefTMB$SBS_refSigs[
    RefTMB$SBS_refSigs$cancerType == "Skin Cancer or Melanoma",
    "COSMIC"
]
W_star <- as.matrix(RefTMB$TMB_SBS_v3.4[, SBS.list])

V1 <- SimData$SingleTumorEx[, "singleV", drop = FALSE]
L1 <- SimData$SingleTumorEx[, "singleL", drop = FALSE]
```

```
colnames(V1) <- colnames(L1) <- "single_tumor"

H_hat <- EstimateSigActivity(V = V1, L = L1, W = W_star)
SigBdn <- CalculateSignatureBurdens(L = L1, W = W_star, H = H_hat$H)
```

This workflow supports targeted-sequencing refitting when the set of signatures is known from a cancer-type-matched catalogue or from a prior cohort-level SATS analysis.

Tests, Workflow Example and Docker

Unit tests are provided in `source/tests/testthat/` for `CalculateSignatureBurdens()`, `EstimateSigActivity()`, `GeneratePanelSize()`, `GenerateVMatrix()` and `GenerateLMatrix()`. To run them locally:

```
cd source
Rscript -e 'testthat::test_dir("tests/testthat")'
```

The repository includes a minimal Nextflow example in `nextflow/example1/`. The example calls `GeneratePanelSize()` on an input `.rda` file containing `genomic_information` and optional `Class`, `SBS_order` and `ref.genome` objects. It is intended as a template for incorporating SATS panel-context generation into standardized workflow systems.

A Dockerfile is also provided for building an R environment with SATS and its core dependencies installed.

Citation and Web Resources

If you use SATS or the targeted-sequencing mutational-signature catalogue, please cite:

Lee et al., "A real-world pan-cancer catalogue of mutational signatures from 111,711 tumors" (submitted).

Interactive catalogue plots are available at https://analysistools.cancer.gov/mutational-signatures/#/catalog/STS. An online signature refitting tool is available at https://analysistools.cancer.gov/mutational-signatures/#/refitting.